

Tutorial/Lab 4 – Generics

Aim

This tutorial/lab aims to further the understanding of generics, from Week 4's lecture.

Exercises

Exercise 4.1: Generic `ArrayList` Reversal

Consider the following method, which reverses the order of elements in an `ArrayList<Integer>`:

```
public static void reverse(ArrayList<Integer> list) {  
    for (int i = 0, j = list.size() - 1; i < j; i++, j--) {  
        Integer temp = list.get(i);  
        list.set(i, list.get(j));  
        list.set(j, temp);  
    }  
}
```

Revise this method so that it can reverse an `ArrayList` containing objects of any type. The method should start with the following identifier:

```
public static <E> void reverse(ArrayList<E> list)
```

Guidelines:

1. Make only the changes necessary to support generic objects.
2. Test your method with an `ArrayList<String>`.
3. Show the list before and after calling the method.

Exercise 4.2: Generic `ArrayList` Shuffling

Consider Exercise 3.1 from last week's lab, in which you are asked to write a method that shuffles an `ArrayList` of Numbers, using a data structure `ArrayList<Number>`. Now we have learned generic type. Your task is to revise the data structure to store generic objects. The `shuffle` method should start with an identifier of the following format:

```
public static <E> void shuffle(ArrayList<E> list)
```

Guidelines:

1. Review the code you developed for Exercise 3.1 last week.

2. Make necessary changes to the codes, to ensure that the `ArrayList` contains generic type of items, and the shuffle method can handle `ArrayList` of different types.
3. Test your shuffle method with an `ArrayList` for **Integer** objects.

Exercise 4.3: Comparable and Generic

1. Based on your answer in Exercise 4.2 above, write a method that returns the largest element in the `ArrayList`. The method should start with the following identifier:

```
public static <E extends Comparable<E>> E max(ArrayList<E> list)
```

2. Use plain English to explain the meaning of the above line:

The method `max` takes an `ArrayList` of _____ and returns _____.

Guidelines for Task 1:

- Finding the largest would require comparisons among the `ArrayList`'s elements. Java has a build-in `compareTo()` method that you can use for this purpose.
- Test your `max` method with an `ArrayList` for `Integer` (not `int`).

Exercise 4.4: Multiple Generics

Read and implement the code below. It has a data structure `Pair` composed of two generic elements, `first` and `second`. These two elements have to be the same type `E`. A `print()` method is developed to output these two elements in a single line.

```
1 public class Pair<E> {
2     public E first; // First element
3     public E second; // Second element
4     /** Constructor: a pair e, f */
5     public Pair(E e, E f) {
6         first = e;
7         second = f;
8     }
9     public static void main(String[] args) {
10        Pair<Integer> p1 = new Pair<>(1, 85);
11        Pair<Integer> p2 = new Pair<>(2, 63);
12        print(p1);
13        print(p2);
14    }
15    public static void print(Pair p) {
16        System.out.println(p.first + " " + p.second);
17    }
18 }
```

After examining the code, we feel that `first` and `second` to be the same type is a too strong requirement. Instead, we may want to store different types, such as `Integer`,

Double, or String (or other objects) in first and second in a single Pair. The code segment below shows an example. Your task is to revise the above code, the Pair data structure specifically, so that it may be used to store different types of objects, such as the example shown below.

```
9   public static void main(String[] args) {
10      Pair<Integer, Double> p1 = new Pair<>(1, 85.5);
11      Pair<Integer, String> p2 = new Pair<>(2, "good");
12      print(p1);
13      print(p2);
14  }
```

Exercise 4.5: Wildcard

The code below has a print method designed to print out all elements of an ArrayList. The type of the elements is <Object>; the argument passed to is <Integer>.

According to the knowledge of last lecture (page 47 of lecture 3), Integer is a subclass of Number which is subclass of Object. It seems that the print method should be able to process Integer arguments. However, the code below would not compile successfully.

Why is this? Give an **explanation** and **make ONE correction** to the code so that it can compile and run successfully.

```
1  import java.util.*;
2
3  public class week4test1{
4      public static void main(String[] args) {
5          ArrayList<Integer> c = new ArrayList<>();
6          c.add(3);
7          c.add(4);
8          c.add(12);
9          print(c);
10     }
11     public static void print(ArrayList<Object> o)
12     {
13         for (Object e : o)
14             System.out.println(e);
15     }
16 }
```

Exercise 4.6: Wildcard Copying of Elements Between Lists

Suppose we want to copy all elements from one list into another list. Consider the incomplete method below:

```
1 public static < T > void copy(ArrayList < T > source, ArrayList < T > target) {  
2     for (T item: source) {  
3         target.add(item);  
4     }  
5 }  
6
```

This version works only when `source` and `target` have exactly the same type parameter. We want a more flexible version.

Your task is to revise the method so that:

- the source list can contain elements of type `T` or any subtype of `T`;
- the target list can contain elements of type `T` or any supertype of `T`.

After revision, the following code should compile and run:

```
1 ArrayList < Integer > source = new ArrayList < > ();  
2 source.add(1);  
3 source.add(2);  
4 source.add(3);  
5  
6 ArrayList < Number > target = new ArrayList < > ();  
7 copy(source, target);  
8 System.out.println(target);  
9 |
```

Guidelines:

1. Revise only the method header.
2. Do not change the loop body.
3. Test using `ArrayList<Integer>` as the source and `ArrayList<Number>` as the target.

Exercise 4.7: Conceptual Understanding of Generics

Please answer the following questions:

1. What is the main advantage of using generics in Java?
2. Why does Java require generic type arguments to be reference types rather than primitive types?
3. What is the difference between:
 - an unbounded wildcard: `?`
 - an upper-bounded wildcard: `? extends T`
 - a lower-bounded wildcard: `? super T`
4. Why is `GenericStack<Integer>` **not** a subclass of `GenericStack<Number>`, even though `Integer` is a subclass of `Number`?
5. What is type erasure, and what effect does it have at runtime?
6. Why is the following not allowed in Java?: `T value = new T();`